

Webtech – Abgabe 2

Modellierung mit MongoDB

Für die Anwendung wurde MongoDB als dokumentenbasierte Datenbank verwendet. Diese Entscheidung war sinnvoll, da die Todo-Objekte aus vielen zusammengehörigen Informationen bestehen. Ein Todo enthält nicht nur Titel und Status, sondern zusätzlich Kommentare, Tags, Checklisten-Einträge, Prioritäten, Änderungsverläufe und Fälligkeitsdaten.

Mit MongoDB konnten diese Daten direkt innerhalb eines Dokuments gespeichert werden. Besonders die eingebetteten Strukturen wie Kommentare, Checklisteneinträge oder History-Einträge passen gut zum dokumentenorientierten Ansatz von MongoDB. Dadurch musste keine komplexe Tabellenstruktur mit vielen Beziehungen erstellt werden.

Ein weiterer Vorteil war die Flexibilität der Datenstruktur. Neue Felder können einfach ergänzt werden, ohne dass die komplette Datenbank geändert werden muss. Das war während der Entwicklung hilfreich, da sich die Struktur der Todos mehrfach erweitert hat.

Zusätzlich passt MongoDB gut zu GraphQL, da die Daten bereits in einer ähnlichen Struktur gespeichert werden wie sie später vom Client abgefragt werden. Dadurch konnten viele Daten direkt weitergegeben werden, ohne große Umwandlungen im Backend.

Nachteilig ist jedoch, dass MongoDB weniger strikte Vorgaben als relationale Datenbanken macht. Dadurch muss stärker darauf geachtet werden, dass Daten korrekt und konsistent gespeichert werden. Außerdem können große Dokumente mit vielen eingebetteten Daten auf Dauer unübersichtlich werden.

Rolle von GraphQL gegenüber Rest (Abgabe 1)

In der ersten Teilabgabe wurde eine klassische REST-API verwendet. Dafür mussten mehrere Endpunkte erstellt werden, beispielsweise zum Laden, Erstellen oder Bearbeiten von Todos.

Mit GraphQL konnte die Kommunikation flexibler gestaltet werden. Der Client kann genau die Daten anfragen, die tatsächlich benötigt werden. Dadurch werden unnötige Datenübertragungen vermieden. Besonders bei den komplexeren Todo-Objekten mit Kommentaren, History-Einträgen und Checklistenelementen war dies ein Vorteil.

Zusätzlich werden bei GraphQL alle Anfragen über einen zentralen Endpoint verarbeitet. Die Struktur des Backends bleibt dadurch übersichtlicher und neue Funktionen können einfacher ergänzt werden.

Praktisch war außerdem die Aufteilung in Queries, Mutations und Subscriptions. Queries wurden zum Lesen der Todos verwendet, Mutations zum Bearbeiten der Daten und Subscriptions für Echtzeit-Aktualisierungen.

Im Vergleich zu REST ist GraphQL jedoch komplexer in der Einrichtung. Das Schema, die Resolver und die Typdefinitionen erhöhen den Entwicklungsaufwand. Für kleinere Anwendungen wäre REST teilweise einfacher umzusetzen gewesen.

Nutzen und Grenzen von Pub/Sub und Subscriptions

Für die Echtzeitkommunikation wurden GraphQL Subscriptions mit einem Pub/Sub-System verwendet. Dadurch können Änderungen an Todos sofort an alle verbundenen Clients übertragen werden.

Wenn beispielsweise ein neues Todo erstellt, bearbeitet oder gelöscht wird, erhalten andere Benutzer die Aktualisierung automatisch, ohne die Seite neu laden zu müssen. Dadurch wirkt die Anwendung moderner und unterstützt eine gleichzeitige Nutzung durch mehrere Benutzer.

Die Subscriptions wurden über die Events ``todoCreated``, ``todoUpdated`` und ``todoDeleted`` umgesetzt. Das Frontend kann dadurch direkt auf Änderungen reagieren und die Anzeige automatisch aktualisieren.

Ein Vorteil dieser Lösung ist die bessere Benutzererfahrung durch Live-Updates. Außerdem zeigt die Anwendung dadurch moderne Webtechnologien wie WebSockets und Echtzeitkommunikation.

Allerdings gibt es auch Grenzen. Das verwendete Pub/Sub-System eignet sich hauptsächlich für kleinere Anwendungen oder Entwicklungsprojekte. Bei größeren Anwendungen mit vielen gleichzeitigen Nutzern wäre eine skalierbarere Lösung notwendig, beispielsweise mit Redis oder einem externen Message-Broker.

Zusätzlich erhöhen Subscriptions die technische Komplexität, da neben normalen HTTP-Anfragen auch dauerhafte WebSocket-Verbindungen verwaltet werden müssen. Fehlerbehandlung und Synchronisation werden dadurch schwieriger.

Insgesamt waren Pub/Sub und Subscriptions jedoch sinnvoll, da sie die Echtzeitfähigkeit der Anwendung ermöglicht und das Multi-User-Konzept unterstützt haben.